CCDSALG Term 3, AY 2025 – 2026

MCO2 Documentation – Social Network (Undirected Graph Application)

I. DECLARATION OF INTELLECTUAL HONESTY & ETHICAL USE OF GENERATIVE AI

1. *We declare that the project we are submitting is the product of our own work augmented with responsible and ethical use of generative AI as encoded in Table 1 and Table 2 below.*
2. *We declare that all AI-assisted contributions have been critically examined by our group. We acknowledge the limitations of Generative AI systems, including possible errors or biases, and accept full responsibility and accountability for all contents in all our project deliverables.*
3. *We also declare that each member COOPERATED_and contributed EQUALLY to the project as detailed in Table 3 below.*
4. *No part of our work was shared with another person outside of our group.*

// Instruction #1: Encode your names, and affix your e-signature to attest to the declaration above.

LUMBANG, Vaughn S04

MANGAHAS, Eowyn S09

TAN, Jean Emmanoel S09

// Instruction #2: Fill-up the column 2 of Table 1 below.

For each of the following tasks, identify the degree of AI use among the following:

- **NONE** - No AI was used for this task
- **SCAFFOLDING** - AI was used to build the foundation for this task, e.g., brainstorming, solution ideation
- **REFINEMENT** - AI was used to review, provide feedback, and revise outputs for this task
- **GENERATION** - AI was used to generate portions of the output for this task

Table 1. Responsible and Ethical Use of AI Details

TASKS	DEGREE OF AI USE & BRIEF EXPLANATION
T1: Understand the project specifications (problem) & assign (approximately) EQUAL WEIGHT responsibilities.	***NONE** - we did not use AI in understanding the problem specs and disseminating the responsibilities.*

Task 2. Design your Undirected Graph data structure.	***NONE*** *- AI was not used to design the undirected graph data structure*
Task 3. Implement the modules including the algorithms for BFS and DFS traversals.	***GENERATION*** *- we used AI to generate an initial version of the code for the graph data structure implementation.*
Task 4. Integrate the modules.	***SCAFFOLDING*** *- AI was used to find reference in how to build the GetBaseName function.*
Task 5. Test & debug your solution exhaustively!	***NONE*** *- AI was not used to test and debug the code.*
Task 6. Document your project.	***NONE*** *- AI was not used to complete the documentation*

// Instruction #3: Fill up Table 2. List explicitly ALL AI tools that you employed, and justify their purpose/role in your project.

Table 2. List of AI Tools Used

AI TOOL USED (embed the link)	**JUSTIFICATION/PURPOSE/ROLE**
Claude	We used this to generate an initial part of the code for the graph data structure.
Gemini	We used this when looking for reference in main.c for getting the base name, so that there is no trailing .txt after the filename.

//… add more lines as you deem necessary

// Instruction #4: Fill up columns 1 and 2 of Table 3.

Table 3. Individual Contributions to the Project

NAME	CONTRIBUTION DETAILS
LUMBANG, Vaughn	Encode details of your contributions for each task. Indicate your role, for example, were you the project manager, were you the (lead) programmer for a certain module, were you the (black box) tester for a certain module, which part of the documentation were you responsible for? etc… Encode NONE if you did not contribute to a particular task. T1: Helped with understanding the instructions and brainstormed on who is assigned to each deliverable. T2: Made the undirected graph structure T3: Made the code for the implementation of the BFS and DFS traversals T4: NONE T5: NONE T6: Helped with completing the documentation
MANGAHAS, Eowyn	T1: Helped with understanding the instructions and brainstormed on who is assigned to each deliverable. T2: NONE T3: helped with making the queue.c and .h. T4: Made the code for main.c T5: Tested and debugged the code. T6: Helped with completing the documentation

TAN, Jean Emmanoel	T1: Helped with understanding the instructions and brainstormed on who is assigned to each deliverable. T2: NONE T3: Made the code for the implementation of the graph data structure. T4: NONE T5: Tested the code T6: Helped with completing the documentation.

II. SUBMISSION CHECKLIST

// Instruction #5: Fill up Column 3 of Table 4 below.

Enumerate all files that you submitted including C source files, header source files, input and output text files, PDF file.

Table 4. File Submission Checklist

FILE	DESCRIPTION
graph.c	C source file of the graph data structure
graph.h	Header source file of the graph data structure
queue.c	C source file of the queues
queue.h	Header source file of the queues
traversal.h	Header source file of the traversals (BFS & DFS)
traversal.c	C source file of the traversals (BFS & DFS)
12-BONUS.c	C source file of the BONUS OPPORTUNITY task
A.txt	Given text files
B.txt	Given text files
C.txt	Given text files
F.txt	Given text files
G.txt	Given text files
H.txt	Given text files
K.txt	Given text files
Y.txt	Given text files
Z.txt	Given text files
main.c	C source file of the main module
stack.c	C source file for stacks
stack.h	Header file for stacks
12.PDF	the PDF file of this document

III. INSTRUCTIONS ON HOW TO COMPILE & RUN THE PROGRAMS

// Instruction #6: Indicate how to compile your source files, and how to RUN your exe files from the COMMAND LINE.

Examples are shown below highlighted in yellow. Replace them accordingly. Make sure that all your group members test what you typed below because I will follow them verbatim (copy/paste as is). I will initially test your solution using a sample input text file that you submitted. Thereafter, I will run it again using my own test data:

- How to compile from the command line

C:\MCO2> **gcc -Wall -o 12Main 12-BONUS.c graph.c**

- How to run from command line

C:\MCO2>**12Main**

Next, answer the following questions:

- Is there a compilation (syntax error) in your codes? (YES or NO). NO

WARNING: the project will automatically be graded with a score of **0** if there is syntax error in any of the submitted source code files. Please make sure that your submission does not have a syntax error.

- Is there any compilation warning in your codes? (YES or NO) NO

WARNING: there will be a 1 point deduction for every unique compiler warning. Please make sure that your submission does not have a compiler warning.

Did you do the BONUS part (YES or NO) YES.

NOTE: Please do NOT submit a solution if it is not working correctly based on the exhaustive testing that you have done.

If you did the BONUS part, indicate below:

- How to compile from the command line

MCO2> **gcc -Wall -o bonus12 12-BONUS.c graph.c**

- How to run from the command line

MCO2> **bonus12**

IV. DISCLOSURE OF ERROR(S)

// Instruction #7: Disclose IN DETAIL what is/are NOT working correctly in your solution.

Please be honest about this. NON-DISCLOSURE will result in severe point deduction! Explain briefly the reason why your group was not able to make it work.

The following are NOT working (buggy):

a. NONE
b. NONE

We were not able to make them work because:

a. NONE
b. NONE

V. SOME IMPLEMENTATION DETAILS

// Instruction #8: Disclose details on your graph data structure and traversal algorithms implementation.

1. How did you implement your Graph data structure? Did you implement it using an Adjacency Matrix or Adjacency List or both? What is the reason behind your choice of graph data structure implementation? Explain briefly (at most 5 sentences).

We implemented it as an Adjacency List, using an array of vertices where each vertex holds a linked list (AdjNode nodes with next pointers) of its neighbors, rather than an N×N adjacency matrix. We chose this because the required Output File #3 format (Diana->Hal->Bruce->Clark->\) is literally a linked-list adjacency list rendered as text, so building the underlying structure the same way meant that the output function could just walk the list and print it with no extra conversion step. It's also more memory-efficient for a social network graph, which is typically sparse, so we only allocate nodes for edges that actually exist instead of reserving space for all 20×20 possible pairs upfront. The array of vertices still gives O(1) index-based access for things like degree lookups and BFS/DFS visited tracking, while the linked list per vertex keeps neighbors in their original input-file order, which the spec requires for File #3. When Output File #4 (the matrix) is needed, it's simply derived on the fly from the adjacency list rather than being the primary storage.

2. How did you implement the DFS and BFS traversals? What other data structures (aside from the graph representation) did you use? Explain your algorithm succinctly.

a. DFS Traversal:

DFS() looks up the start vertex's index via FindVertexIndex, zeroes out a visited[] array sized MAX_VERTICES, then hands off to DFSRecursion().

DFSRecursion() does the classic recursive DFS pattern:

1. Mark the current vertex visited and append it to output[].
2. Walk its adjacency list (AdjNode linked list) and collect all unvisited neighbors into a local neighbors[] array.
3. Sort that neighbor array alphabetically by label using sortNeighbors() (a simple bubble sort comparing g->vertices[...].label with strcmp).
4. Recurse into each neighbor in that sorted order, skipping any that got visited in the meantime (since visiting one neighbor's subtree could have visited another).

The alphabetical sort is what makes traversal order deterministic and reproducible regardless of the order edges were inserted into the adjacency list.

b. BFS Traversal:

BFS() follows the same setup (find start index, reset visited[]), then uses a Queue (from queue.h) instead of the call stack:

1. Mark the start vertex visited, enqueue it.
2. While the queue isn't empty: dequeue a vertex, append it to output[].
3. Collect its unvisited neighbors, marking them visited immediately (important — this prevents the same vertex from being enqueued twice via two different in-progress vertices).
4. Sort those neighbors alphabetically (same sortNeighbors() helper), then enqueue them in that order.

Data structures used (besides the graph)

- Queue (queue.h) — a fixed-capacity circular/array-based queue (QUEUE_CAPACITY = 20) with front/count bookkeeping, exposing InitQueue, IsQueueEmpty, Enqueue, Dequeue. Used only by BFS.
- visited[] — a plain int array of size MAX_VERTICES, used by both traversals to avoid revisiting/re-enqueuing vertices.
- output[] — caller-supplied array that both functions fill in traversal order, with *count tracking how many vertices were written.
- neighbors[] — a local scratch array (size MAX_VERTICES) used each time a vertex's adjacency list is scanned, so neighbors can be sorted before being processed/enqueued.
- Implicitly, the call stack acts as DFS's stack (via recursion) rather than an explicit stack structure.

VI. SELF-EVALUATION

// Instruction #8: Fill-up Table 6 below.

Refer to the rubric in the project specs. It is suggested that you do an individual self-assessment first. Thereafter, compute the average evaluation for your group, and encode it below.

Table 5. Self-Evaluation

Criteria	AVE. OF SELF-ASSESSMENT
1. Output Text File #1 (Set of Vertices & Edges)	10 (max. 10 points)
2. Output Text File #2 (Vertices With Degrees)	10 (max. 10 points)
3. Output Text File #3 (Adj. Matrix Visualization)	12.5 (max. 12.5 points)
4. Output Text File #4 (Adj. List Visualization)	12.5 (max. 12.5 points)
5. Output Text File #5 (BFS Traversal)	20 (max. 20 points)
6. Output Text File #6 (DFS Traversal)	20 (max. 20 points)
7. Documentation	10 (max. 10 points)
8. Compliance	5 (max. 5 points)
9. Bonus :-) // ALL-OR-NOTHING	10 (max. 10 points)
	TOTAL:100 **/100**

NOTE: The evaluation that the instructor will give is not necessarily going to be the same as what you indicated above. The self-assessment is for your own reference only…

– The End –

サルバドル・フロランテ